

6장)_합성곱 신경망(CNN)(2)

이미지 분류를 위한 신경망

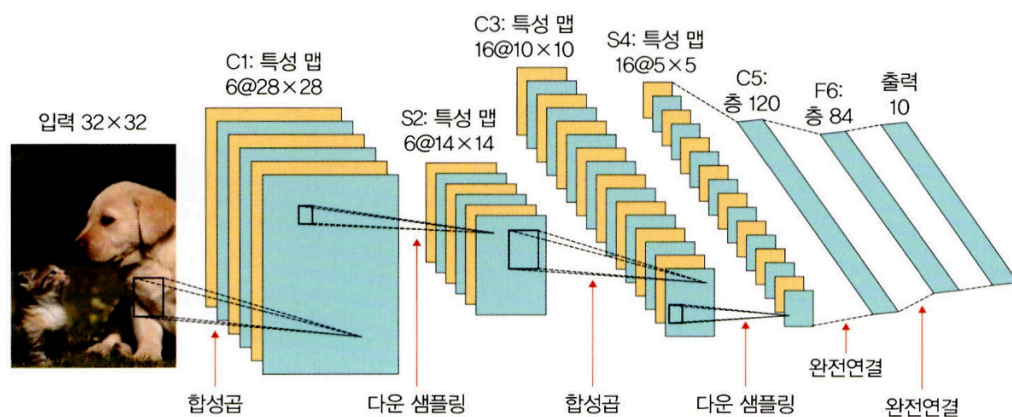
- 입력 데이터로 이미지를 사용한 분류(classification) : 특정 대상이 영상 내에 존재하는지 여부를 판단하는 것

LeNet-5

- 합성곱 신경망이라는 개념을 최초로 안 르쿤(Yann LeCun)이 개발한 구조
 - 1995년 안 르쿤 등이 수표에 손글씨 숫자를 인식하는 딥러닝 구조 LeNet-5를 발표했는데, 그것이 현재 CNN의 초석이 됨
- LeNet-5는 합성곱(convolutional)과 다운 샘플링(sub-sampling)(혹은 풀링)을 반복적으로 거치면서 마지막에 완전연결층에서 분류를 수행

고전적인 LeNet-5 구조를 기반으로 한 CNN 아키텍처 연산 과정

▼ 그림 6-1 LeNet-5



- 입력: 32 × 32 이미지 (예: 흑백 MNIST)
 - 보통 LeNet은 32x32 크기의 입력을 기준 (MNIST는 원래 28x28인데 32x32로 패딩해서 쓰기도 함)
- (cf) C : convolution layer, S : pooling layer, F : fully connected layer

- **C1: Convolution layer** ⇒ 5 X 5 합성곱 연산 후 28 X 28 크기의 특성 맵(feature map) 6개를 생성
 - 커널: 5x5
 - 입력: 32x32
 - 패딩: X (없음)
 - 스트라이드: 1
 - 출력 크기 계산: $\frac{32-5}{1+1} = 28 \rightarrow 28 \times 28$
 - 출력 채널(필터 수): 6개 → **28x28x6**

- **S2: Pooling layer (Subsampling)** ⇒ 다운 샘플링하여 특성 맵 크기를 14 X 14로 줄임
 - 방식: 보통 2x2 평균 또는 최대 풀링
 - 스트라이드: 2 → 크기가 절반으로 줄어듦
 - 28x28 → **14x14**
 - 채널 수는 그대로 6개 → **14x14x6**

- **C3: Convolution layer** ⇒ 5 X 5 합성곱 연산하여 10 X 10 크기의 특성 맵 16개를 생성
 - 커널: 5x5
 - 입력: 14x14
 - 출력: $\frac{14-5}{1+1} = 10 \rightarrow 10 \times 10$

- 출력 채널 수: 16개 → **10×10×16**

- **S4: Pooling layer** ⇒ 다운 샘플링하여 특성 맵 크기를 5 X 5로 줄임

- 10×10 → **5×5** (2×2 풀링)
- 채널 수 유지: 16개 → **5×5×16**

- **C5: Convolution layer** ⇒ 5 X 5 합성곱 연산하여 1 X 1 크기의 특성 맵 120개를 생성

- 커널: 5×5
- 입력: 5×5
- 출력: $\frac{5-5}{1+1} = 1 \rightarrow 1 \times 1$
- 출력 채널 수: 120개 → **1×1×120**

⇒ 이 레이어는 사실상 완전연결층처럼 작동 → 이미지 한 조각에서 정보 다 뽑아내는 것

- **F6: Fully Connected layer** ⇒ 완전연결층으로 C5의 결과를 유닛(unit)(또는 노드) 84개에 연결시킴

- 입력: 120개 (C5의 출력)
- 출력: 84개 유닛에 연결

⇒ 이걸 그냥 Dense Layer

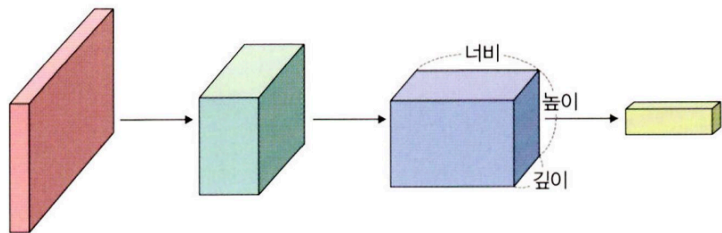
AlexNet

- ImageNet 영상 데이터베이스를 기반으로 한 화상 인식 대회인 'ILSVRC 2012'에서 우승한 CNN 구조

AlexNet 구조

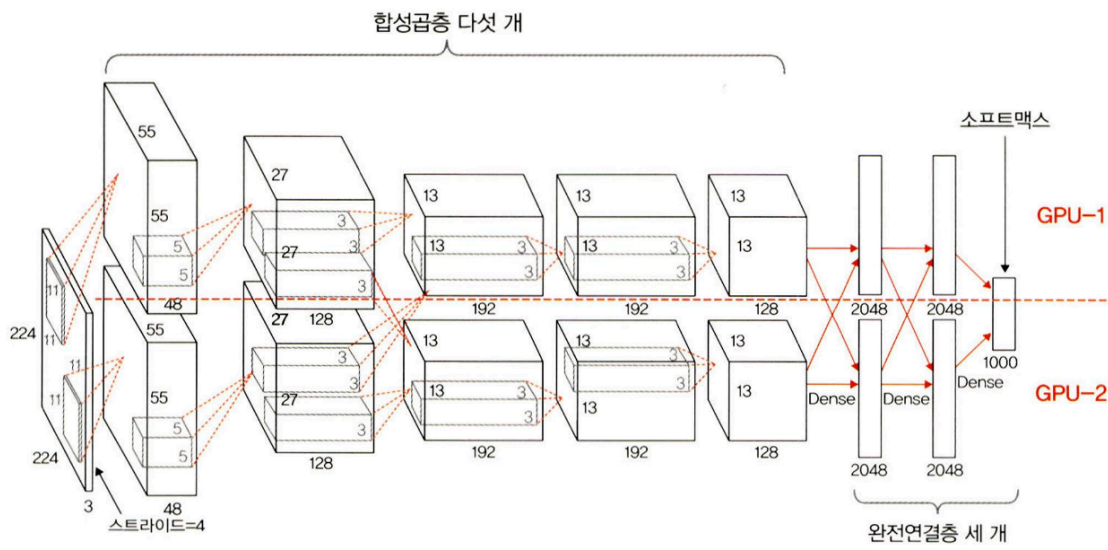
AlexNet 구조를 이해하기 위해 CNN 구조 먼저 알아보기

▼ 그림 6-8 CNN 구조



- 그림과 같이 3차원 구조를 가짐(이미지를 다루기 때문에 기본적으로 3차원 데이터)
 - 이미지 크기를 나타내는 너비(width), 높이(height) 뿐만 아니라 깊이(depth)를 가짐
 - 보통 색상이 많은 이미지는 R/G/B 성분 세 개를 갖기 때문에 시작이 3이지만, 합성곱을 거치면서 특성 맵이 만들어지고 이에 따라 중간 영상의 깊이가 달라짐

♥ 그림 6-9 AlexNet 구조



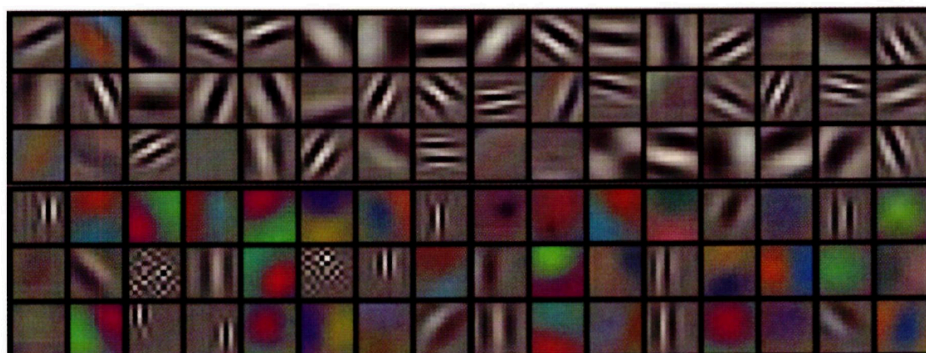
- AlexNet은 합성곱층 5개와 완전연결층 3개로 구성되어 있음
 - AlexNet의 합성곱층에서 사용된 활성화 함수는 ReLU
 - 맨 마지막 완전연결층은 카테고리 1000개를 분류하기 위해 Softmax 활성화 함수 사용
- GPU 2개를 기반으로한 병렬 구조인 점을 제외하면 LeNet-5와 크게 다르지 않음

♥ 표 6-2 AlexNet 구조 상세

계층 유형	특성 맵	크기	커널 크기	스트라이드	활성화 함수
이미지	1	227×227	—	—	—
합성곱층	96	55×55	11×11	4	렐루(ReLU)
최대 풀링층	96	27×27	3×3	2	—
합성곱층	256	27×27	5×5	1	렐루(ReLU)
최대 풀링층	256	13×13	3×3	2	—
합성곱층	384	13×13	3×3	1	렐루(ReLU)
합성곱층	384	13×13	3×3	1	렐루(ReLU)
합성곱층	256	13×13	3×3	1	렐루(ReLU)
최대 풀링층	256	6×6	3×3	2	—
완전연결층	—	4096	—	—	렐루(ReLU)
완전연결층	—	4096	—	—	렐루(ReLU)
완전연결층	—	1000	—	—	소프트맥스(softmax)

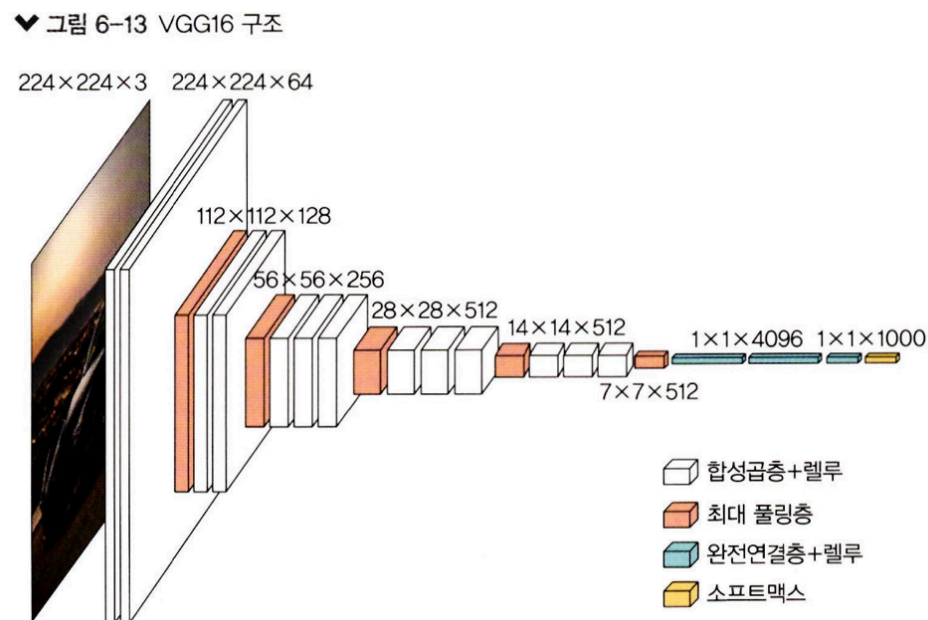
- 첫번째 합성곱층 커널의 크기는 11 x 11 × 3(입력 채널 수만큼 커널을 가짐)이며, 스트라이드 4로 적용하여 특성맵을 96(필터 개수가 96 개라 출력 채널 96개 생성됨)개 생성하기 때문에 55 x 55× 96의 출력을 가짐
 - $$\text{Output size} = \left\lfloor \frac{\text{Input size} - \text{Kernel size}}{\text{Stride}} \right\rfloor + 1$$
- 첫번째 계층을 거치면서 GPU-1에서는 주로 컬러와 상관없는 정보를 추출하기 위한 커널이 학습되고, GPU-2에서는 주로 컬러와 관련된 정보를 추출하기 위한 커널이 학습됨
- 각 GPU의 결과

♥ 그림 6-10 AlexNet GPU-1 · 2 적용 결과



VGGNet

- **VGGNet은 합성곱층의 파라미터 수를 줄이고 훈련 시간을 개선하려고 탄생**
 - 즉, 네트워크를 깊게 만드는 것이 성능에 어떤 영향을 미치는지 확인하고자 나온 것
 - 깊이의 영향만 최대한 확인하고자 합성곱층에서 사용하는 필터/커널의 크기를 가장 작은 3x3으로 고정
- 네트워크 계층의 총 개수에 따라 여러 유형 (VGG16, VGG19 등)이 있음
- VGG16



- 파라미터 : 1억 3300만 개
- 합성곱 커널 크기 : 3 x 3
- 최대 풀링 커널 크기 : 2 x 2
- 스트라이드 : 2
- 출력 : 64개의 224 x 244 특성 맵 → 224 x 224 × 64 생성
- 마지막 16번째 계층 제외하고 모두 ReLU 활성화 함수 적용

▼ 표 6-3 VGG16 구조 상세

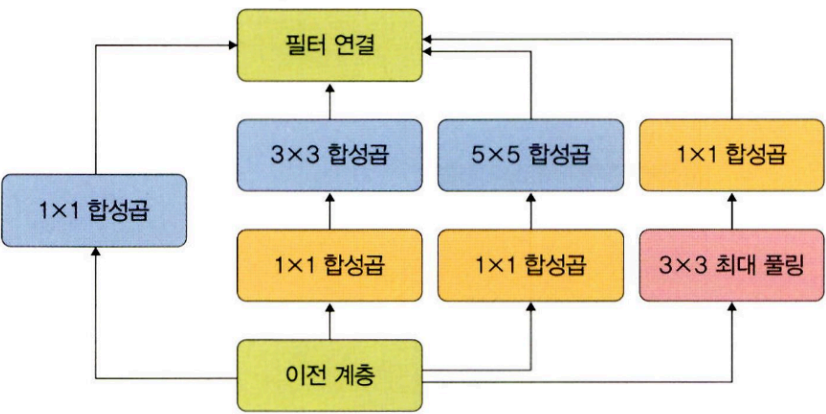
계층 유형	특성 맵	크기	커널 크기	스트라이드	활성화 함수
이미지	1	224×224	—	—	—
합성곱층	64	224×224	3×3	1	렐루(ReLU)
합성곱층	64	224×224	3×3	1	렐루(ReLU)
최대 풀링층	64	112×112	2×2	2	—
합성곱층	128	112×112	3×3	1	렐루(ReLU)
합성곱층	128	112×112	3×3	1	렐루(ReLU)
최대 풀링층	128	56×56	2×2	2	—
합성곱층	256	56×56	3×3	1	렐루(ReLU)
합성곱층	256	56×56	3×3	1	렐루(ReLU)
합성곱층	256	56×56	3×3	1	렐루(ReLU)
합성곱층	256	56×56	3×3	1	렐루(ReLU)
최대 풀링층	256	28×28	2×2	2	—
합성곱층	512	28×28	3×3	1	렐루(ReLU)
합성곱층	512	28×28	3×3	1	렐루(ReLU)
합성곱층	512	28×28	3×3	1	렐루(ReLU)
합성곱층	512	28×28	3×3	1	렐루(ReLU)
최대 풀링층	512	14×14	2×2	2	—

계층 유형	특성 맵	크기	커널 크기	스트라이드	활성화 함수
합성곱층	512	14×14	3×3	1	렐루(ReLU)
합성곱층	512	14×14	3×3	1	렐루(ReLU)
합성곱층	512	14×14	3×3	1	렐루(ReLU)
합성곱층	512	14×14	3×3	1	렐루(ReLU)
최대 풀링층	512	7×7	2×2	2	—
완전연결층	—	4096	—	—	렐루(ReLU)
완전연결층	—	4096	—	—	렐루(ReLU)
완전연결층	—	1000	—	—	소프트맥스(softmax)

GoogLeNet

- GoogLeNet은 주어진 하드웨어 자원을 최대한 효율적으로 이용하면서 학습 능력은 극대화할 수 있는 깊고 넓은 신경망
- 깊고 넓은 신경망을 위해 인셉션(inception) 모듈 추가
 - 효율적으로 추출하기 위해 1x1, 3x3, 5x5의 합성곱 연산을 각각 수행
 - 3x3 최대 풀링은 입력과 출력의 높이와 너비가 같아야 하므로 풀링 연산에서는 드물게 패딩을 추가해야함
- 결과적으로 GogLeNet에 적용된 해결 방법은 희소 연결(sparse connectivity)
 - CNN은 합성곱, 풀링, 완전연결층들이 서로 밀집(dense)하게 연결되어 있음
 - 빽빽하게 연결된 신경망 대신 관련성(correlatioin)이 높은 노드끼리만 연결하는 방법을 희소 연결이라고 함
 - 연산량이 적어지며 과적합도 해결할 수 있음
- 인셉션 모델의 4가지 연산

▼ 그림 6-23 GoogLeNet의 인셉션 모듈



- 1x1 합성곱
 - 1x1 합성곱 + 3x3 합성곱
 - 1x1 합성곱 + 5x5 합성곱
 - 3x3 최대 풀링(maxpooling) + 1x1 합성곱(convolutional)
- 딥러닝을 이용하여 ImageNet과 같은 대회에 참여하거나 서비스를 제공하려면 대용량 데이터를 학습해야함
 - 심층 신경망의 아키텍처에서 계층이 넓고(뉴런이 많고) 깊으면(계층이 많으면)
 - 인식률은 좋아지지만
 - 과적합이나 기울기 소멸 문제(vanishing gradient problem)을 비롯한 학습 시간 지연과 연산 속도 등의 문제
 - 특히, 합성곱 신경망에서 이런 문제들이 자주 나타남 → GoogLeNet(혹은 Inception)으로 이러한 문제 해결 가능